

Guía Técnica: Control de Versiones con Git y GitHub

Introducción al Desarrollo Colaborativo

1. ¿Qué es el Control de Versiones?

El control de versiones es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, de modo que sea posible recuperar versiones específicas más adelante.

Existen dos tipos principales:

- **Sistemas Centralizados (CVCS):** Un único servidor contiene todos los archivos versionados (ej. SVN).
- **Sistemas Distribuidos (DVCS):** Cada usuario tiene una copia completa del repositorio en su máquina local. **Git** es el estándar actual de este tipo.

2. Importancia del Control de Versiones

La importancia radica en la seguridad, la trazabilidad y la colaboración.

Ejemplo: Imagine que está desarrollando una nueva funcionalidad para una aplicación de escritorio. Tras horas de trabajo, accidentalmente borra una carpeta crítica o introduce un error que rompe todo el sistema. Sin control de versiones, perdería el progreso. Con Git, simplemente puede realizar un `revert` o volver al `commit` anterior en segundos, salvando horas de trabajo.

3. Conceptos Básicos

- **Repositorio (Repo):** Carpeta donde Git guarda el historial.
- **Commit:** Una captura o "foto" del estado de tus archivos en un momento dado.
- **Staging Area (Index):** Zona intermedia donde preparas los archivos antes de confirmar un commit.
- **Remote:** Una versión del proyecto alojada en internet (ej. GitHub).

4. Funcionamiento de Ramas (Branching)

Las ramas permiten divergir de la línea de desarrollo principal (`main`) para trabajar en nuevas funciones o corregir errores sin afectar el código estable. Una rama es esencialmente un puntero móvil a uno de los commits.

Cuando la funcionalidad en la rama es estable, se realiza un **merge** (fusión) para integrar los cambios de vuelta a la rama principal.

5. Flujo de Trabajo Estándar

1. **Clonar:** Descargar un repositorio remoto a tu máquina local con `git clone`.
2. **Trabajar:** Modificar archivos en tu directorio de trabajo.
3. **Stage:** Preparar los cambios con `git add`.
4. **Commit:** Guardar los cambios localmente con `git commit`.
5. **Pull Request (PR):** En GitHub, proponer que tus cambios sean revisados e integrados al repositorio original.

6. Comandos Esenciales

Comando	Descripción
<code>git init</code>	Inicia un nuevo repositorio local.
<code>git status</code>	Muestra el estado de los archivos (modificados, en stage).
<code>git add .</code>	Agrega todos los cambios al área de preparación.
<code>git commit -m "mensaje"</code>	Registra una nueva versión con un comentario.
<code>git branch</code>	Lista las ramas o crea una nueva.
<code>git checkout -b [nombre]</code>	Crea una rama y se cambia a ella inmediatamente.
<code>git log --oneline</code>	Muestra el historial de commits resumido.

7. Actividad Práctica: Laboratorio de Ramas

Objetivo: Al finalizar este laboratorio, podrá utilizar comandos de Git para trabajar con ramas en un repositorio local.

Pasos a seguir:

1. **Inicialización:** Cree una carpeta y ejecute `git init`.
2. **Primer Archivo:** Cree un archivo `readme.txt`, agréguelo con `git add` y haga su primer `git commit`.
3. **Nueva Rama:** Cree una rama llamada `desarrollo` usando `git branch desarrollo`.
4. **Navegación:** Cambie a la rama recién creada con `git checkout desarrollo`.
5. **Modificación:** Edite el archivo, vea los cambios con `git status` y realice un nuevo commit en esta rama.
6. **Historial:** Use `git log` para observar que el historial en `desarrollo` es diferente al de `main`.
7. **Reversión:** Practique el uso de `git revert [hash-del-commit]` para deshacer un cambio de forma segura.
8. **Fusión (Merge):** Regrese a la rama `main` y fusione los cambios de `desarrollo` usando `git merge desarrollo`.